Λύση
Καταρχάς θα πρέπει να αποσαφηνίσουμε τη διεπαφή της συνάρτησης. Σύμφωνα με την εκφώνηση, η συνάρτηση λαμβάνει έναν μονοδιάστατο πίνακα ακέραιων. Επομένως, η λίστα παραμέτρων είναι σαφής. Επίσης, σύμφωνα με την εκφώνηση, η συνάρτηση επιστρέφει το μέγιστο στοιχείο του πίνακα. Επομένως, επιστρέφει έναν ακέραιο. Ο κώδικας 7.17 παρουσιάζει τη συνάρτηση.

```
public static int max(int[] tbl) { //Κώδικας 7.17
    int max = tbl[0];
    int idx = 0;
    for (int i = 1; i < tbl.length; i++) {
        if (tbl[i] > max) {
            idx = i;
            max = tbl[i];
        }
    }
    return idx;
}
```

Κώδικας 7.17 *Συνάρτηση εύρεσης της μέγιστης τιμής σε πίνακα ακέραιων*

Για την υλοποίηση της συνάρτησης χρησιμοποιούμε δύο βοηθητικές μεταβλητές, την max και την idx. Η max χρησιμεύει για την αποθήκευση της τιμής του μεγαλύτερου στοιχείου και η idx για την αποθήκευση της θέσης του μεγαλύτερου στοιχείου. Η idx αρχικοποιείται ώστε να αναφέρεται στην αρχή του πίνακα και η max στην τιμή του στοιχείου στην θέση 0.

Στη συνέχεια, σε μια επαναληπτική διαδικασία ελέγχονται ένα προς ένα τα υπόλοιπα στοιχεία του πίνακα. Κάθε φορά που το υπό έλεγχο στοιχείο είναι μεγαλύτερο από την τιμή της max ενημερώνεται τόσο η max όσο και η idx. Στο τέλος του βρόχου for, η max είναι ενημερωμένη με την τιμή του μεγαλύτερου στοιχείου του πίνακα και η idx με τη θέση του.

Προσέξτε πως η for ξεκινάει από 1 και όχι από 0 όπως συνήθως. Αν ξεκινούσε από 0, στο πρώτο επαναληπτικό βήμα θα εξέταζε αν το στοιχείο στη θέση 0 είναι μεγαλύτερο από την τιμή της max. Η max όμως έχει αρχικοποιηθεί στην τιμή του στοιχείου στη θέση 0. Επομένως, πρόκειται για περιττή σύγκριση.

Επίσης, ενδιαφέρον έχει πως η συνάρτηση επιστρέφει τη θέση της μεγαλύτερης τιμής και όχι την ίδια την τιμή. Πράγματι, είναι πιο χρήσιμη η θέση παρά η τιμή καθώς αν έχω τη θέση, εύκολα μπορώ να προσπελάσω την τιμή.

Τέλος, ας σημειωθεί πως αν στον πίνακα υπάρχουν περισσότερα από ένα μέγιστα στοιχεία, η υλοποίηση του κώδικα 7.17 επιστρέφει τη θέση του πρώτου από τα μέγιστα.

7.9.2 Εξίσωση β’ βαθμού

Να υλοποιηθεί συνάρτηση που επιστρέφει τη λύση εξίσωσης β’ βαθμού.

Λύση
Καταρχάς θα πρέπει να δούμε πώς θα μοντελοποιήσουμε το πρόβλημα, δηλαδή στην προκειμένη περίπτωση να αποφασίσουμε ποια θα είναι η διεπαφή της συνάρτησης. Γνωρίζουμε από τα μαθηματικά πώς η εξίσωση β’ βαθμού έχει τη μορφή

$$ax^2 + bx + c = 0, με\ a, b, c\ \in R\ και\ a \neq 0$$

Επομένως, η εξίσωση ορίζεται από τους συντελεστές της. Αυτούς πρέπει να περάσουμε παραμετρικά στη συνάρτηση. Τι όμως θα επιστρέφει η συνάρτηση; Όπως γνωρίζουμε, οι λύσεις εξαρτώνται από την τιμή της διακρίνουσας. Αν η διακρίνουσα, Δ, είναι μεγαλύτερη από το 0, τότε έχουμε δύο διακριτές πραγματικές λύσεις, αν η διακρίνουσα είναι ίση με το 0, τότε έχουμε μια κοινή λύση και αν η διακρίνουσα είναι μικρότερη από το 0, η εξίσωση δεν έχει λύση.

Μία αντιμετώπιση είναι να επιστρέφουμε έναν πίνακα πραγματικών μεγέθους 2. Στην περίπτωση που $\Delta > 0$, τότε στην πρώτη θέση του πίνακα θα έχουμε τη μία ρίζα και στη δεύτερη την άλλη, αν $\Delta = 0$ και στις δύο θέσεις θα έχουμε την κοινή ρίζα, τέλος αν $\Delta < 0$, τότε και στις δύο θέσεις θα τοποθετήσουμε την τιμή Double.NaN.

Βέβαια όλα αυτά για να τα κάνουμε θα πρέπει να υπολογίσουμε τη διακρίνουσα. Παρόλο που ο χρήστης της συνάρτησης θα πληροφορηθεί για το πρόσημο της διακρίνουσας, δεν θα γνωρίζει την ακριβή τιμή της. Μία εναλλακτική προσέγγιση είναι η συνάρτησή μας να επιστρέφει πίνακα μήκους 3, όπου στη θέση 0 να τοποθετεί την τιμή της διακρίνουσας. Στον κώδικα 7.18 δίνεται αυτή η τελευταία εκδοχή της συνάρτησης.

```
static double[] triwnymo(double a, double b, double c) { // Κώδικας 7.18
    if (a == 0) {
        throw new RuntimeException();
    }
    double[] rVal = new double[3];
    double d = b * b - 4 * a * c;
    rVal[0] = d;
    if (d < 0) {
        rVal[1] = rVal[2] = Double.NaN;
        return rVal;
    }
    rVal[1] = (-b + Math.sqrt(d)) / (2 * a);
    rVal[2] = (-b - Math.sqrt(d)) / (2 * a);
    return rVal;
}
```

Κώδικας 7.18 *Συνάρτηση λύσης εξίσωσης β' βαθμού*

Σε αυτήν την περίπτωση η εξίσωση μπορεί να κληθεί όπως δείχνει ο κώδικας 7.19.

```
static void klisiTriwnymo() { // Κώδικας 7.19
    double[] solution = triwnymo(2, 5, -1);
    if (solution[0] < 0) {
        System.out.println("Η Εξίσωση σεν έχει λύση στο R");
    } else {
        System.out.println("χ1=" + solution[1] + " x2=" + solution[2]);
    }
}
```

Κώδικας 7.19 *Κλήση της συνάρτησης triwnymo*

Μία εναλλακτική κλήση δίνεται στον κώδικα 7.20

```
static void klisiTriwnymo2() { // Κώδικας 7.20
    double[] solution = triwnymo(2, 5, -1);
    if (Double.isNaN(solution[1])) {
        System.out.println("Η Εξίσωση σεν έχει λύση στο R");
    } else {
        System.out.println("x1=" + solution[1] + " x2=" + solution[2]);
    }
}
```

Κώδικας 7.20 *Εναλλακτική κλήση της συνάρτησης triwnymo*

Προσοχή όμως δεν είναι σωστή η κλήση

```
if (solution[1]==Double.NaN)….
```

καθώς εξορισμού η έκφραση Double.NaN==Double.NaN είναι ψευδής.
Σε μια εναλλακτική προσέγγιση μπορούμε να αναπαραστήσουμε το τριώνυμο ως έναν πίνακα πραγματικών μήκους 3. Η συνάρτηση μπορεί να υπερφορτωθεί εύκολα ώστε να υποστηρίζεται και αυτή η αναπαράσταση του τριωνύμου, όπως δείχνει ο κώδικας 7.21.

```
static double[] triwnymo(double[] syn) { // Κώδικας 7.21
    return triwnymo(syn[0], syn[1], syn[2]);
}
```

Κώδικας 7.21 *Υπερφόρτωση της συνάρτησης triwnymo*

Σε αυτήν την περίπτωση, γίνεται η αναπαράσταση εξισώσεων β' βαθμού ευκολότερη, όπως δείχνει ο κώδικας 7.22.

```
double[] ex1 = {2, 5, -1}, ex2 = {4, 4, -9};
double[] solution1 = triwnymo(ex1);
double[] solution2 = triwnymo(ex2);
```

Κώδικας 7.22 *Αναπαράσταση εξισώσεων β βαθμού ως double[]*

7.9.3 Fibonacci

Να αναπτυχθεί συνάρτηση που επιστρέφει τον ν-οστό όρο της ακολουθίας Fibonacci.

Λύση
Καταρχάς θα πρέπει να δούμε ποιος ακριβώς είναι ο ορισμός της ακολουθίας Fibonacci. Στο διαδίκτυο μπορούμε εύκολα να τον βρούμε. Αν συμβολίσουμε τον ν-οστό με Fn, τότε ο ορισμός της ακολουθίας έχει ως εξής:

$$n = 0 \rightarrow F_n = 0$$
$$n = 1 \rightarrow F_n = 1$$
$$n > 1 \rightarrow F_n = F_{n-1} + F_{n-2}$$

Επομένως, η συνάρτησή μας θα πρέπει να λαμβάνει μία ακέραια παράμετρο, μπορεί όμως να επιστρέφει ένα long, δεδομένου πως η τιμή του ν-οστού όρου είναι σημαντικά μεγαλύτερη από το n. Ο κώδικας 7.23 δίνει μια υλοποίηση της συνάρτησης.

```
static long fibonacci(int fibIdx) { //Κώδικας 7.23
    if (fibIdx < 0) {
        throw new RuntimeException();
    } else if (fibIdx <= 1) {
        return fibIdx;
    }
    int n = 2;
    long current = 1, previous = 1, beforePrevious;
    while (++n <= fibIdx) {
        beforePrevious = previous;
        previous = current;
        current = beforePrevious + previous;
    }
    return current;
}
```

Κώδικας 7.23 *Υλοποίηση της συνάρτησης Fibonacci*

Καθώς δεν υπάρχει αρνητικός όρος της ακολουθίας, αν η συνάρτηση κληθεί με παράμετρο μικρότερη του μηδενός, σημαίνει πως κάποιο λάθος έχει συμβεί. Διαφορετικά αν η παράμετρος fibIdx είναι μικρότερη ή ίση με το 1, δηλαδή αν είναι 0 ή 1, βάσει ορισμού, επιστρέφεται η ίδια η τιμή της παραμέτρου. Αν παρόλα αυτά δεν επιστρέψει η συνάρτηση σημαίνει ότι κλήθηκε με παράμετρο μεγαλύτερη του ένα, οπότε η τιμή του ν-οστού όρου πρέπει να υπολογιστεί ως το άθροισμα των δύο προηγούμενων όρων. Η μεταβλητή n αναπαριστά την τάξη του όρου και ξεκινά από το 2, οι δε μεταβλητές current, previous και beforePrevious αναπαριστούν τις τιμές του ν-οστού όρου, του προηγούμενού του και του όρου πριν τον προηγούμενο του ν-οστού. Ο βρόχος while επισκέπτεται έναν-έναν τους όρους της ακολουθίας έως ότου φτάσει στον όρο που εκφράζει η παράμετρος fibIdx. Σε κάθε βήμα αυτής της επαναληπτικής διαδικασίας ενημερώνονται κατάλληλα οι προηγούμενοι όροι. Με την έξοδο από την while, η current είναι ενημερωμένη με την τιμή του όρου στη θέση fibIdx.

7.9.4 Λεκτικό αριθμού

Να υλοποιηθεί συνάρτηση static String lektiko(int num) που δέχεται μια ακέραια παράμετρο από 0 έως 999 και επιστρέφει τη διατύπωσή της στα Ελληνικά.

Λύση

Σύμφωνα με την εκφώνηση και με τη διεπαφή που δίνεται μια κλήση όπως η lektiko(125) θα πρέπει να επιστρέψει την αλφαριθμητική σειρά “εκατόν είκοσι πέντε”. Εδώ έχουμε δύο θέματα να μας απασχολήσουν. Πρώτον, πόσα και ποια λεκτικά μας είναι απαραίτητα για να σχηματίσουμε την απαιτούμενη διατύπωση όλων των αριθμών από 0 έως 999 και δεύτερον πώς θα εξάγουμε πόσες εκατοντάδες έχουμε, πόσες δεκάδες και πόσες μονάδες. Προσέξτε πως αν διαιρέσετε έναν ακέραιο με το 100 (ακέραια διαίρεση), τότε λαμβάνεται τον αριθμό εκατοντάδων που περιέχει. Αν στη συνέχεια βγάλετε τις εκατοντάδες, μένει το υπόλοιπο του ακέραιου. Για παράδειγμα: Έστω num=475, h ο αριθμός των εκατοντάδων και r το υπόλοιπο, τότε h=num/100=4, r=num-h*100=75. Στη συνέχεια διαιρούμε το υπόλοιπο με το 10 για να λάβουμε τον αριθμό των δεκάδων. Τέλος, οι μονάδες είναι το υπόλοιπο μετά την αφαίρεση των δεκάδων. Ο κώδικας 7.24 επιστρέφει το λεκτικό ενός ακέραιου από το 0 έως το 999.

```
static String lektiko(int num) { //Κώδικας 7.24
    if (num < 0 || num > 999) {
        throw new RuntimeException();
    }
    if (num == 0) {
        return "μηδέν";
    }
    int ekatontades = num / 100;
    num -= ekatontades * 100;
    int dekades = num / 10;
    num -= dekades * 10;
    String hundreds = "", tens = "", units = "";
    switch (ekatontades) {
        case 0:
            break;
        case 1:
            hundreds = "εκατόν";
            break;
        case 2:
            hundreds = "διακόσια";
            break;
        case 3:
            hundreds = "τριακόσια";
            break;
        case 4:
```

```
            hundreds = "τετρακόσια";
            break;
        case 5:
            hundreds = "πεντακόσια";
            break;
        case 6:
            hundreds = "εξακόσια";
            break;
        case 7:
            hundreds = "επτακόσια";
            break;
        case 8:
            hundreds = "οκτακοσια";
            break;
        case 9:
            hundreds = "εννιακόσια";
            break;
    }
    switch (dekades) {
        case 0:
            break;
        case 1:
            tens = "δέκα";
            break;
        case 2:
            tens = "είκοσι";
            break;
        case 3:
            tens = "τριάντα";
            break;
        case 4:
            tens = "σαράντα";
            break;
        case 5:
            tens = "πενήντα";
            break;
        case 6:
            tens = "εξήντα";
            break;
        case 7:
            tens = "εβδομήντα";
            break;
        case 8:
            tens = "ογδόντα";
            break;
        case 9:
            tens = "ενενήντα";
            break;
    }
    switch (num) {
        case 0:
            break;
        case 1:
            if (tens.equals("δέκα")) {
                tens = "έντεκα";
            } else {
                units = "ένα";
```

```
                }
                break;
            case 2:
                if (tens.equals("δέκα")) {
                    tens = "δώδεκα";
                } else {
                    units = "δύο";
                }
                break;
            case 3:
                units = "τρία";
                break;
            case 4:
                units = "τέσσερα";
                break;
            case 5:
                units = "πέντε";
                break;
            case 6:
                units = "έξη";
                break;
            case 7:
                units = "επτά";
                break;
            case 8:
                units = "οκτώ";
                break;
            case 9:
                units = "εννέα";
                break;
        }
        String rVal = hundreds + (tens != "" ? " " : "") + tens + (units !=
"" ? " " : "") + units;
        //return rVal;
        return rVal.trim();
    }
```

Κώδικας 7.24 *Μετατροπή ακέραιου σε λεκτικό*

Προσέξτε την έκφραση tens!=””?” ”:””. Σημαίνει πως αν η μεταβλητή tens έχει πάρει τιμή διάφορη από ““, τότε επιστρέφουμε ένα διάστημα που θα διαχωρίσει τις εκατοντάδες από τις δεκάδες. Αν όμως η tens παραμένει ““, τότε σημαίνει πως δεν έχουμε δεκάδες, επομένως δεν υπάρχει λόγος να προσθέσουμε το αντίστοιχο διάστημα. Ίδια λογική ισχύει και για την units!=””?” ”:””.

7.9.5 Κλάση MyArrays

Να υλοποιηθεί η κλάση MyArrays που περιλαμβάνει τις ακόλουθες στατικές συναρτήσεις χωρίς να χρησιμοποιηθεί η κλάση Arrays.

```
public static int seqSearch(int[] t, int sE)
```

Πραγματοποιεί σειριακή αναζήτηση στον t για την τιμή sE. Επιστρέφει τη θέση στον t της sE εφόσον αυτή βρεθεί, διαφορετικά -1.

```
public static int seqSearch(String[] t, String sE)
```

Πραγματοποιεί σειριακή αναζήτηση στον t για την τιμή sE. Επιστρέφει τη θέση στον t της sE εφόσον αυτή βρεθεί, διαφορετικά -1.

```
public static int cntOccurances(int[] t, int sE)
```

Μετράει και επιστρέφει το πλήθος τιμών sE στον t.

```
public static String toString(int[] t)
```

Επιστρέφει μια String αναπαράσταση του t.

```
public static String toString(int[][] t)
```

Επιστρέφει μια String αναπαράσταση του t.

```
public static int[] reverse(int[] t)
```

Επιστρέφει έναν πίνακα ακέραιων που περιέχει τα στοιχεία του t με αντεστραμμένη διάταξη.

```
public static int sum(int[] t)
```

Επιστρέφει το άθροισμα των στοιχείων του t.

```
public static int max(int[] t)
```

Επιστρέφει τη θέση του μέγιστου στοιχείου του t.

```
public static boolean equals(int[] tA, int[] tB)
```

Επιστρέφει true αν κάθε στοιχείο του tA είναι και στοιχείο του tB και κάθε στοιχείο του tB είναι και στοιχείο του tA.

```
public static boolean equals(String[] tA, String[] tB)
```

Επιστρέφει true αν κάθε στοιχείο του tA είναι και στοιχείο του tB και κάθε στοιχείο του tB είναι και στοιχείο του tA.

```
public static int binarySearch(int[] t, int sE)
```

Πραγματοποιεί δυαδική αναζήτηση στον t για την τιμή sE. Αν η τιμή βρεθεί, επιστρέφει τη θέση της στον t, διαφορετικά επιστρέφει -1.

```
public static void swap(int[] t, int idx1, int idx2)
```

Αντιμεταθέτει το στοιχείο στη θέση idx1 του t με το στοιχείο στη θέση idx2.

```
public static int idxOfMin(int[] array, int startIdx)
```

Επιστρέφει τη θέση του ελάχιστου στοιχείου του t από τη θέση startIdx και μετά.

```
public static void sort(int[] t)
```

Ταξινομεί κατ'αύξουσα αριθμητική σειρά των πίνακα t.

Λύση
Ο κώδικας 7.25 παραθέτει μια λύση της άσκησης και ακολουθούν σχόλια.

```
public class MyArrays {

    public static int seqSearch(int[] t, int sE) {
        for (int i = 0; i < t.length; i++) {
            if (t[i] == sE) {
                return i;
            }
        }
        return -1;
    }

    public static int seqSearch(String[] t, String sE) {
        for (int i = 0; i < t.length; i++) {
            if (t[i].equals(sE)) {
                return i;
            }
        }
        return -1;
    }

    public static int cntOccurances(int[] t, int sE) {
        int cnt = 0;
        for (int e : t) {
            if (e == sE) {
                cnt++;
            }
        }
        return cnt;
    }

    public static String toString(int[] t) {
        if (t.length == 0) {
            return "[]";
        }
        String s = "[";
        for (int i = 0; i < t.length; i++) {
            s += t[i];
            if (i < t.length - 1) {
                s += ", ";
            }
        }
        s += "]";
        return s;
    }

    public static String toString(int[][] t) {
        if (t.length == 0) {
            return "[]";
        }
        String s = "[";
```